Microservices Notes: Service Discovery & OpenFeign

1. Why do we need Service Discovery?

In a traditional application, we might call another service using a fixed IP and port:

```
Order Service
      |
      ↓
http://192.168.1.10:8081
      |
      ↓
Payment Service
```

This becomes a problem in microservices because:

- A service can have multiple instances.
- Instances can be created or removed dynamically.
- IP addresses can change.
- Services can restart.
- Auto-scaling can increase/decrease instances.
- We don't want to hardcode IP addresses and ports.

For example:

```
Payment Service

Instance 1 → 10.0.0.10:8081
Instance 2 → 10.0.0.11:8081
Instance 3 → 10.0.0.12:8081
```

Instead of the Order Service knowing all these addresses, we use Service Discovery.

2. What is a Service Registry?

A Service Registry is a central repository that maintains information about available service instances.

For example:

```
Service Registry

SERVICE             HOST          PORT
-------------------------------------------------
PAYMENT-SERVICE     10.0.0.10    8081
PAYMENT-SERVICE     10.0.0.11    8081
ORDER-SERVICE       10.0.0.20    8082
USER-SERVICE        10.0.0.30    8083
```

The registry basically maintains: which services are available and where their instances are running.

Interview definition: A Service Registry is a centralized repository that stores information about available instances of microservices, such as their service name, host, port, and status.

Examples: Netflix Eureka, Consul, Apache Zookeeper.

Kubernetes provides its own service discovery mechanism rather than requiring Eureka in many deployments.

3. What is Service Discovery?

Service Discovery is the process/mechanism used by one service to find another service dynamically.

Suppose:

```
Order Service
      |
      | "I need PAYMENT-SERVICE"
      ↓
Discovery Client
      |
      ↓
Service Registry
      |
      | "These Payment instances are available"
      ↓
Payment instances
```

The Order Service doesn't need to know `10.0.0.10`, `10.0.0.11`, `10.0.0.12` — it can refer to the logical service name: `PAYMENT-SERVICE`.

4. Service Registry vs Service Discovery

This is a very important interview question.

Service Registry	Service Discovery
Stores service information	Finds service information
Acts like a service catalog	Acts like a mechanism/process
Knows available service instances	Helps clients locate those instances
Stores host, port, status, metadata, etc.	Retrieves available instances
Example: Eureka Server	Often performed using a Discovery Client

Easy way to remember: - Registry = Address Book - Discovery = Looking up the address

For example:

```
Address Book
     ↓
PAYMENT-SERVICE → 10.0.0.10:8081
```

Finding that address is service discovery.

5. Important: What is a Discovery Client?

A Discovery Client is a component/library running with the microservice that communicates with the Service Registry.

It can typically:

- Register the microservice.
- Send heartbeats/renewals.
- Retrieve information about other services.

For example:

```
Order Service
      |
      ↓
Discovery Client
      |
      ↓
Eureka Server
```

The Discovery Client helps Order Service discover Payment Service.

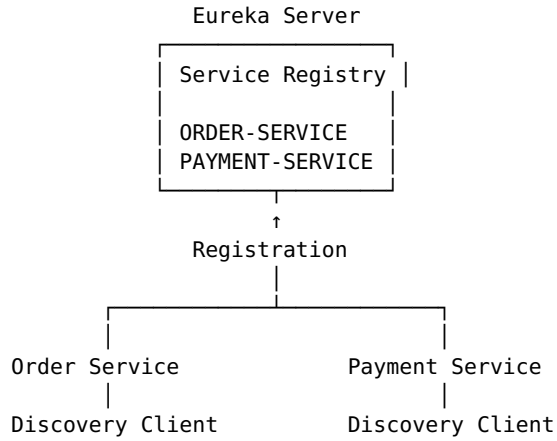
Important clarification — don't say: *"Discovery Client sends all requests."*

Instead: *"The Discovery Client communicates with the service registry to register the service and discover other service instances. The actual business request is sent to the discovered service instance by an HTTP client, Feign client, WebClient, RestClient, load balancer, or another communication mechanism."*

6. Complete Flow

Suppose we have: `ORDER-SERVICE`, `PAYMENT-SERVICE`, `EUREKA SERVER`.

The complete process is:



Now suppose Order Service wants to call Payment Service:

```
Order Service
```

```
     |
     | 1. Discover PAYMENT-SERVICE
     ↓
Discovery Client
     |
     | 2. Ask registry
     ↓
Eureka Server
     |
     | 3. Return available instances
     ↓
Discovery Client
     |
     | 4. Select/resolve an instance
     ↓
Payment Service
     |
     | 5. Actual business request
     ↓
Response
```

The exact division between discovery, load balancing, and HTTP request handling depends on the Spring Cloud setup.

7. Service Registration

When a microservice starts, it needs to tell the registry: *"I am available."*

```
Payment Service
       |
       | REGISTER
       ↓
Eureka Server
```

It might register:

```
Service Name: PAYMENT-SERVICE
Host: 10.0.0.10
Port: 8081
Instance ID: payment-001
Status: UP
```

The registry now knows:

```
PAYMENT-SERVICE
       ↓
10.0.0.10:8081
```

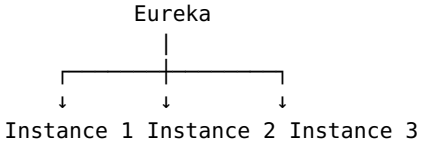
8. Multiple Instances

Suppose Payment Service is scaled to three instances:

```
PAYMENT-SERVICE

Instance 1 → 10.0.0.10:8081
Instance 2 → 10.0.0.11:8081
Instance 3 → 10.0.0.12:8081
```

All three register themselves:



The registry maintains all of them.

9. Service Discovery Request

Now Order Service wants Payment Service. Instead of `http://10.0.0.10:8081`, it uses the logical service name `PAYMENT-SERVICE`.

The Discovery Client asks: *"Give me instances of PAYMENT-SERVICE"*

The registry responds:

```
10.0.0.10:8081
10.0.0.11:8081
10.0.0.12:8081
```

Now an instance must be selected — this is where load balancing comes into the picture.

10. Service Discovery vs Load Balancing

These are different concepts.

Service Discovery answers: *"Where are the available Payment Service instances?"*

```
10.0.0.10
10.0.0.11
10.0.0.12
```

Load Balancing answers: *"Which available instance should handle this request?"*

```
10.0.0.10 ← selected
```

So:

```
Service Discovery
       ↓
Find instances
       ↓
Load Balancer
       ↓
Select instance
       ↓
Actual request
```

---

11. Dependencies in Eureka Server

Eureka Server

This is the main dependency. It turns the application into a Service Registry. Think of Eureka Server as a directory/address book where all microservices register themselves.

```
Eureka Server
     |
     ├── Accounts Service
     ├── Cards Service
     ├── Loans Service
     └── Customer Service
```

So when one service wants to find another service, it can ask Eureka.

Config Client

Config Client allows the Eureka Server to get its configuration from a central Config Server, instead of putting everything directly inside the Eureka Server (`port = 8070`, `hostname = localhost`).

```
Eureka Server
      ↓
"Give me my configuration"
      ↓
Config Server
      ↓
Configuration
```

Spring Boot Actuator

Actuator is mainly used for monitoring and health checks — is Eureka Server UP, is the application healthy, what is the application status.

Simple meaning: Actuator = Monitoring and Health Check.

12. `@EnableEurekaServer`

This annotation is used only in the Eureka Server application:

```
@EnableEurekaServer
```

It tells Spring: *"This application is going to act as a Eureka Server."*

You do not add this annotation to Accounts, Cards, Loans, etc.

```
Eureka Server
@EnableEurekaServer
       ↓
Acts as Service Registry
```

Other microservices use `Eureka Discovery Client`, not `@EnableEurekaServer`.

13. What do other Microservices need?

For example, Accounts Service, Cards Service, and Loans Service need the Eureka Discovery Client dependency:

`spring-cloud-starter-netflix-eureka-client`

This allows them to communicate with Eureka Server.

14. Eureka Server Configuration

`server.port: 8070`

```yaml
server:
  port: 8070
```

Eureka Server will run on port 8070 — accessible at `localhost:8070`.

`eureka.instance.hostname: localhost`

```yaml
eureka:
  instance:
    hostname: localhost
```

This tells Eureka: *"My hostname is localhost."* Commonly used when developing on your own computer.

`fetchRegistry: false`

```yaml
eureka:
  client:
    fetchRegistry: false
```

Eureka Server does not need to download the service registry, because Eureka Server itself *is* the registry. (Think of a library: the library doesn't need to borrow its own catalog.)

`registerWithEureka: false`

```yaml
eureka:
  client:
    registerWithEureka: false
```

Eureka Server does not register itself as a normal microservice, because it is already the registry.

```
Eureka Server → registerWithEureka = false
Normal Microservice → registerWithEureka = true
```

`serviceUrl.defaultZone`

```yaml
serviceUrl:
  defaultZone: http://localhost:8070/eureka/
```

This tells the application where the Eureka Server is located. The other microservices use this address to connect to Eureka.

15. Eureka Client Configuration

This is the configuration used by Accounts, Cards, Loans, etc.

`preferIpAddress: true`

```yaml
eureka:
  instance:
```

```yaml
    preferIpAddress: true
```

Tells Eureka: *"Register my service using my IP address instead of my hostname"* (e.g. `10.0.0.15` instead of `accounts-service.company.com`). Useful in Docker/cloud deployments.

`fetchRegistry: true`

```yaml
eureka:
  client:
    fetchRegistry: true
```

Means: *"I want to get the list of other services from Eureka."* Accounts Service can then get information about Cards, Loans, and Customer Service.

Simple meaning: Download/get the list of registered services from Eureka.

`registerWithEureka: true`

```yaml
eureka:
  client:
    registerWithEureka: true
```

Means: *"Register my service with Eureka."* When Accounts Service starts:

```
Accounts Service
       |
       | "I am available"
       ↓
Eureka Server
```

Eureka stores information such as `Accounts Service, IP: 10.0.0.15, Port: 8080` — now other services can find Accounts Service.

`serviceUrl.defaultZone` (in microservices)

```yaml
eureka:
  client:
    serviceUrl:
      defaultZone: http://localhost:8070/eureka/
```

Tells the microservice: *"Connect to Eureka Server at this address."*

```
Accounts Service
       |
       ↓
http://localhost:8070/eureka/
       |
       ↓
Eureka Server
```

16. Application Info Properties

`info.app.name`

```yaml
info:
  app:
    name: "accounts"
```

Gives the application a name — here, `accounts`. Helps identify the application.

`info.app.description`

```yaml
info:
  app:
    description: "Eazy Bank Accounts Application"
```

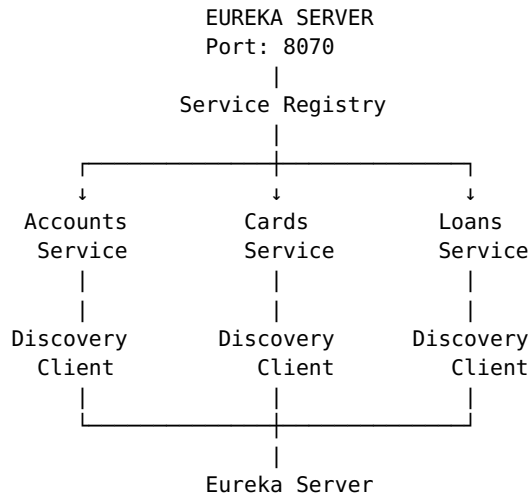
A short description of the application — *what is this application?*

`info.app.version`

```yaml
info:
  app:
    version: "1.0.0"
```

The version of the application currently running (later you might release `1.1.0`, `2.0.0`, etc.), helping identify which version is deployed.

17. Complete Picture



When Accounts Service starts:

```
Accounts Service
       |
       | Register myself
       ↓
Eureka Server
```

When Accounts Service wants to find Cards Service:

```
Accounts Service
       |
       | "Where is Cards Service?"
       ↓
Discovery Client
       |
       ↓
Eureka Server
       |
       | "Cards Service is here"
```

```
      ↓
Cards Service
```

18. Very Important Interview Summary

Remember these 5 points:

1. **Eureka Server** — Stores information about all registered services.
2. **Discovery Client** — Helps a microservice register itself and discover other services.
3. **`registerWithEureka: true`** — Register myself with Eureka.
4. **`fetchRegistry: true`** — Get the list of other services from Eureka.
5. **`@EnableEurekaServer`** — Turn this application into a Eureka Server.

Easy Memory Trick:

```
REGISTER  → "I am here!"          → Eureka Server
FETCH     → "Who else is here?"   → Eureka Server
DISCOVER  → "Where is Cards Service?" → Eureka Server
ACTUAL REQUEST → "Call Cards Service" → Cards Service
```

And the most important distinction: the Discovery Client helps **FIND** the service. It is not the same thing as the component that performs the actual business request. The actual call can be made through Feign, RestClient, WebClient, an HTTP client, etc., often with load balancing involved.

OpenFeign Customer-Details Flow — Code Walkthrough

1. Add the OpenFeign dependency

`pom.xml`:

```xml
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-openfeign</artifactId>
        </dependency>
```

This pulls in the machinery that lets an interface become a working REST client — no manual `RestTemplate`/`WebClient` calls needed.

2. Turn on Feign clients in the main class

```java
        @SpringBootApplication
        @EnableFeignClients
        @EnableJpaAuditing(auditAwareRef = "auditAwareImpl")
        @EnableConfigurationProperties(value =
{AccountsContactInfoDto.class})
        public class AccountsApplication {
            public static void main(String[] args) {
                SpringApplication.run(AccountsApplication.class, args);
            }
        }
```

`@EnableFeignClients` tells Spring to scan for interfaces annotated `@FeignClient` and generate real HTTP-calling implementations for them at startup.

3. `CardsFeignClient` — declarative client for the Cards service

```
        @FeignClient("cards")
        public interface CardsFeignClient {

            @GetMapping(value = "/api/fetch", consumes = "application/json")
            public ResponseEntity<CardsDto> fetchCardDetails(@RequestParam
String mobileNumber);

        }
```

- `@FeignClient("cards")` — `"cards"` is the *service ID* of the Cards microservice (resolved via Eureka/service discovery), not a URL.
- The method signature here **must mirror** the real endpoint on the Cards side — same path, same params, same return type — because Feign just proxies the call through. That's exactly what the actual `CardsController` looks like on the Cards service:

```
        @GetMapping("/fetch")
        public ResponseEntity<CardsDto> fetchCardDetails(
                @RequestParam @Pattern(regexp = "(^$|[0-9]{10})", message =
"Mobile number must be 10 digits")
                String mobileNumber) {
            CardsDto cardsDto = iCardsService.fetchCard(mobileNumber);
            return ResponseEntity.status(HttpStatus.OK).body(cardsDto);
        }
```

4. `LoansFeignClient` — same pattern, for Loans

Not shown directly in your screenshots, but its usage (`loansFeignClient.fetchLoanDetails(mobileNumber)`) confirms it mirrors `CardsFeignClient` exactly:

```
        @FeignClient("loans")
        public interface LoansFeignClient {

            @GetMapping(value = "/api/fetch", consumes = "application/json")
            public ResponseEntity<LoansDto> fetchLoanDetails(@RequestParam
String mobileNumber);

        }
```

5. `CardsDto` (lives in the `accounts` service's own `dto` package)

```
        @Schema(name = "Cards", description = "Schema to hold Card
information")
        @Data
        public class CardsDto {
```

```
            @NotEmpty(message = "Mobile Number can not be a null or empty")
            @Pattern(regexp = "(^$|[0-9]{10})", message = "Mobile Number
must be 10 digits")
            @Schema(description = "Mobile Number of Customer", example =
"4354437687")
            private String mobileNumber;

            // additional fields (cardNumber, cardType, totalLimit, etc.)
likely exist
            // but weren't visible in the screenshot
        }
```

This is `accounts`'s own copy of the shape it expects Cards to return — the Feign client deserializes the Cards response straight into it.

6. `CustomerDetailsDto` — the aggregated response shape

```
        @Schema(name = "CustomerDetails", description = "Schema to hold
Customer, Account, Cards and Loans information")
        @Data
        public class CustomerDetailsDto {

            private String name;
            private String email;
            private String mobileNumber;

            @Schema(description = "Account details of the Customer")
            private AccountsDto accountsDto;

            @Schema(description = "Loans details of the Customer")
            private LoansDto loansDto;

            @Schema(description = "Card details of the Customer")
            private CardsDto cardsDto;
        }
```

This is the one object the client actually receives — one field per microservice's contribution.

7. `ICustomersService` — the contract

```
        package com.eazybytes.accounts.service;

        import com.eazybytes.accounts.dto.CustomerDetailsDto;

        public interface ICustomersService {

            /**
             * @param mobileNumber - Input Mobile Number
             * @return Customer Details based on a given mobileNumber
             */
            CustomerDetailsDto fetchCustomerDetails(String mobileNumber);
        }
```

8. `CustomersServiceImpl` — where the aggregation actually happens

```
        @Service
        @AllArgsConstructor
        public class CustomersServiceImpl implements ICustomersService {

            private AccountsRepository accountsRepository;
            private CustomerRepository customerRepository;
            private CardsFeignClient cardsFeignClient;
            private LoansFeignClient loansFeignClient;

            @Override
            public CustomerDetailsDto fetchCustomerDetails(String
mobileNumber) {
                // 1. Look up the customer + their account from the local DB
                Customer customer =
customerRepository.findByMobileNumber(mobileNumber).orElseThrow(
                        () -> new ResourceNotFoundException("Customer",
"mobileNumber", mobileNumber)
                );
                Accounts accounts =
accountsRepository.findByCustomerId(customer.getCustomerId()).orElseThrow(

                        () -> new ResourceNotFoundException("Account",
"customerId", customer.getCustomerId().toString())
                );

                // 2. Build the base DTO from local data
                CustomerDetailsDto customerDetailsDto =
                        CustomerMapper.mapToCustomerDetailsDto(customer, new
CustomerDetailsDto());
                customerDetailsDto.setAccountsDto(
                        AccountsMapper.mapToAccountsDto(accounts, new
AccountsDto()));

                // 3. Reach out over the network to the Loans microservice
via Feign
                ResponseEntity<LoansDto> loansDtoResponseEntity =
loansFeignClient.fetchLoanDetails(mobileNumber);

customerDetailsDto.setLoansDto(loansDtoResponseEntity.getBody());

                // 4. (Cards fetch would follow the identical pattern via
cardsFeignClient)

                return customerDetailsDto;
            }
        }
```

Your last screenshot cuts off right after the Loans call, so the Cards fetch + final `return` may still need to be added — I've noted that as step 4 above so you don't lose the thread.

9. `CustomerMapper`

```
        public class CustomerMapper {
```

```
            public static CustomerDto mapToCustomerDto(Customer customer,
CustomerDto customerDto) {
                customerDto.setName(customer.getName());
                customerDto.setEmail(customer.getEmail());
                customerDto.setMobileNumber(customer.getMobileNumber());
                return customerDto;
            }

            public static CustomerDetailsDto
mapToCustomerDetailsDto(Customer customer, CustomerDetailsDto
customerDetailsDto) {
                customerDetailsDto.setName(customer.getName());
                customerDetailsDto.setEmail(customer.getEmail());

customerDetailsDto.setMobileNumber(customer.getMobileNumber());
                return customerDetailsDto;
            }
        }
```

10. `CustomerController` — the entry point

```
        package com.eazybytes.accounts.controller;

        @Tag(
                name = "REST API for Customers in EazyBank",
                description = "REST APIs in EazyBank to FETCH customer
details"
        )
        @RestController
        @RequestMapping(path = "/api", produces = {
MediaType.APPLICATION_JSON_VALUE })
        @Validated
        public class CustomerController {

            private final ICustomersService iCustomersService;

            public CustomerController(ICustomersService iCustomersService) {
                this.iCustomersService = iCustomersService;
            }

            @GetMapping("/fetchCustomerDetails")
            public ResponseEntity<CustomerDetailsDto> fetchCustomerDetails(
                    @RequestParam
                    @Pattern(regexp = "(^$|[0-9]{10})", message = "Mobile
number must be 10 digits")
                    String mobileNumber) {
                CustomerDetailsDto customerDetailsDto =
iCustomersService.fetchCustomerDetails(mobileNumber);
                return
ResponseEntity.status(HttpStatus.OK).body(customerDetailsDto);
            }
        }
```

- Constructor injection of `ICustomersService` — standard, testable DI.
- `@Pattern(regexp = "(^$|[0-9]{10})")` — validates the mobile number is either empty or exactly 10 digits before the service layer even runs.

End-to-end request flow

1. `GET /api/fetchCustomerDetails?mobileNumber=...` hits `CustomerController`.
2. Controller delegates to `CustomersServiceImpl.fetchCustomerDetails(...)`.
3. Service pulls `Customer` + `Accounts` from the local database.
4. Service calls `LoansFeignClient` and (once finished) `CardsFeignClient` — each of these is a normal-looking Java interface call, but Feign turns it into an actual HTTP request to the Loans/Cards microservice.
5. Everything gets merged into one `CustomerDetailsDto` and returned to the caller.